

Correction étudiant — TD/TP 1

Premiers pas du C vers le C++
Initialisation, références et `const`
MAM4 — Programmation C++

Cette correction reprend les six exercices dans l'ordre du TD. Les programmes complets sont écrits en C++20 et doivent compiler avec :

```
g++ -std=c++20 -Wall -Wextra -pedantic fichier.cpp -o programme
```

Les lignes volontairement incorrectes sont clairement signalées. Le texte exact d'un diagnostic peut varier selon le compilateur.

Exercice 1 — Un fichier C n'est pas toujours du C++

Le programme du TD compile en C avec :

```
gcc -std=c17 -Wall -Wextra -pedantic fichier.c -o programme
```

En revanche, sa copie `fichier.cpp` est refusée par le compilateur C++ :

```
error: invalid conversion from 'void*' to 'int*'
```

Réponse à la question b. `malloc` renvoie un `void*`, c'est-à-dire un pointeur qui ne précise pas le type de l'objet pointé. Le C autorise ici sa conversion implicite en `int*`, mais le C++ la refuse. Ajouter une conversion explicite ferait disparaître le diagnostic sans adopter une gestion plus naturelle des ressources en C++.

Allocation manuelle en C++ :

```
int main() {  
    int n{8};  
    int* values{new int[n]};  
  
    values[0] = 42;  
  
    delete[] values;  
}
```

Version avec un conteneur C++ :

```
#include <vector>  
  
int main() {  
    int n{8};  
    std::vector<int> values(n);  
  
    values[0] = 42;  
    // Aucune libération explicite.  
}
```

Dans la première version, il faut appeler `delete[]` soi-même. Dans la seconde, l'objet `std::vector` gère son stockage.

C	C++ manuel	C++ usuel ici
<code>malloc</code>	<code>new[]</code>	<code>std::vector<int></code>
<code>free</code>	<code>delete[]</code>	aucune libération explicite

À retenir. En C++ moderne, on préfère généralement un objet comme `std::vector` à la gestion manuelle de la mémoire.

Exercice 2 — Initialisation et conversions

```
#include <cmath>
#include <iostream>

int main() {
    double x{};
    double y{};

    std::cout << "x ? ";
    std::cin >> x;
    std::cout << "y ? ";
    std::cin >> y;

    double moyenne{(x + y) / 2.0};
    double distance{std::abs(x - y)};

    std::cout << "moyenne = " << moyenne << '\n';
    std::cout << "distance = " << distance << '\n';
}
```

`double x{};` initialise `x` à 0.0. `<iostream>` fournit `std::cin` et `std::cout`; `<cmath>` fournit ici `std::abs`.

Pour les entrées 4 et 10, la moyenne vaut 7 et la distance vaut 6.

```
int a = 2.9; // Compile : a vaut 2.
int b{2.9}; // Erreur de compilation.
```

1. `a` compile; `b` est refusée.
2. La conversion vers `int` perd la partie fractionnaire. La valeur devient 2 : il s'agit d'une troncature, pas d'un arrondi.
3. Dans un calcul scientifique, le refus de `b` évite qu'une perte d'information passe silencieusement dans les résultats.

On appelle cela une *narrowing conversion*, c'est-à-dire une conversion réductrice pouvant perdre de l'information.

À retenir. L'initialisation avec `{}` aide le compilateur à détecter certaines pertes d'information.

Exercice 3 — Découvrir les références

En C, l'appel utilisait les adresses des variables de sortie :

```
double lo;
double hi;
minmax(values, n, &lo, &hi);
```

En C++, les deux sorties peuvent être reçues par référence. On suppose dans cet exercice que $n > 0$ et que le tableau contient au moins n éléments valides.

```
#include <iostream>

void minmax(const double* values,
            int n,
            double& minimum,
            double& maximum) {
    minimum = values[0];
    maximum = values[0];

    for (int i{1}; i < n; ++i) {
        if (values[i] < minimum) {
            minimum = values[i];
        }

        if (values[i] > maximum) {
            maximum = values[i];
        }
    }
}

int main() {
    double values[]{4.5, -2.0, 8.0, 1.5, 3.0};
    int n{5};
    double lo{};
    double hi{};

    minmax(values, n, lo, hi);

    std::cout << "minimum = " << lo << '\n';
    std::cout << "maximum = " << hi << '\n';
}
```

Sortie :

```
minimum = -2
maximum = 8
```

1. La fonction modifie `lo` et `hi` : ce sont ses deux résultats.
2. Elle ne doit pas modifier les valeurs du tableau.
3. `const double* values` permet de lire les éléments sans les modifier à travers ce pointeur. Les références `minimum` et `maximum` ne sont pas `const`, car la fonction doit les affecter.

Différence entre copie et référence :

```
double x{3.0};
double r{x};
r = 7.0;
// x vaut toujours 3.0
```

```
double x{3.0};
double& r{x};
r = 7.0;
// x vaut maintenant 7.0
```

Dans le premier cas, `r` est un nouvel objet. Dans le second, `r` et `x` sont deux noms pour le même objet.

Fonction renvoyant une référence :

```
#include <iostream>

int& toto(int& value)
{
    return value;
}

int main()
{
    int z{4};

    toto(z) = 18;

    std::cout << "z = " << z << '\n';
    std::cout << &z << '\n';
    std::cout << &toto(z) << '\n';
}
```

La sortie contient `z = 18`. Les deux adresses sont identiques : `toto(z)` est un autre accès à `z`, pas un nouvel entier. Le retour `int&` permet donc de placer l'appel à gauche de `=`. Avec un retour par valeur `int`, cette affectation est refusée, car l'appel produit une valeur indépendante et non un objet modifiable de l'appelant.

À retenir. Une référence est un autre nom donné à un objet existant. Une fonction peut également renvoyer cet alias.

Exercice 4 — T, T& ou const T& ?

Pour choisir un paramètre :

T	la fonction reçoit sa propre valeur
T&	la fonction travaille sur l'objet existant et peut le modifier
const T&	la fonction travaille sur l'objet existant sans le modifier via cette référence

Les signatures attendues sont :

```
double mean(const Sample& s);
void center(Sample& s);
void print(const Sample& s);
double square(double x);
void multiply_by_two(double& x);
```

Programme complet, en supposant que `s.values` n'est pas vide :

Réponse. Dans cet exercice, `std::string` et `std::vector` sont des types fournis. Leur fonctionnement sera étudié à la séance suivante; le choix demandé porte seulement sur `T`, `T&` et `const T&`.

```
#include <iostream>
#include <string>
#include <vector>

struct Sample {
    std::string name;
    std::vector<double> values;
};

double mean(const Sample& s) {
    double sum{};
    for (double value : s.values) {
        sum += value;
    }
    return sum / s.values.size();
}

void center(Sample& s) {
    double m{mean(s)};
    for (double& value : s.values) {
        value -= m;
    }
}

void print(const Sample& s) {
    std::cout << s.name << " :";
}
```

```

    for (double value : s.values) {
        std::cout << ' ' << value;
    }
    std::cout << '\n';
}

double square(double x) {
    return x * x;
}

void multiply_by_two(double& x) {
    x *= 2.0;
}

int main() {
    Sample s{"test", {2.0, 4.0, 6.0}};

    print(s);
    std::cout << "moyenne = " << mean(s) << '\n';
    center(s);
    print(s);

    double x{5.0};
    std::cout << "square(x) = " << square(x) << '\n';
    multiply_by_two(x);
    std::cout << "x = " << x << '\n';
}

```

Sortie :

```

test : 2 4 6
moyenne = 4
test : -2 0 2
square(x) = 25
x = 10

```

Dans `center`, le `&` est indispensable :

- `for (double value : s.values)` modifierait seulement une copie temporaire de chaque élément ;
- `for (double& value : s.values)` permet de modifier les éléments réels du vecteur.

Pour un petit type tel que `double`, le passage par valeur est naturel quand aucune modification de l'appelant n'est demandée. Il ne faut donc pas mettre mécaniquement tous les paramètres en `const T&`.

À retenir. Avant de choisir le type de paramètre, demander : la fonction doit-elle modifier l'objet ?

Exercice 5 — Ce code compile-t-il ?

Analyse ligne par ligne :

Ligne	Compile ?	Explication
<code>double x{2.0};</code>	oui	<code>x</code> est modifiable
<code>const double y{3.0};</code>	oui	<code>y</code> ne peut plus être modifié
<code>double& r1{x};</code>	oui	<code>r1</code> est un alias modifiable de <code>x</code>
<code>double& r2{y};</code>	non	cette référence permettrait de modifier <code>y</code> malgré son <code>const</code>
<code>const double& r3{y};</code>	oui	lecture possible, modification interdite via <code>r3</code>
<code>r1 += 1.0;</code>	oui	modifie <code>x</code> , qui passe de 2 à 3
<code>r3 += 1.0;</code>	non	essaie de modifier via une référence en lecture seule

Diagnostics possibles :

```

error: binding reference of type 'double&'
      to 'const double' discards qualifiers
error: assignment of read-only reference 'r3'

```

Programme compilable après avoir commenté les deux lignes incorrectes :

```
#include <iostream>

int main() {
    double x{2.0};
    const double y{3.0};

    double& r1{x};
    // double& r2{y};
    const double& r3{y};

    r1 += 1.0;
    // r3 += 1.0;

    std::cout << x << '\n';
    std::cout << y << '\n';
    std::cout << r3 << '\n';
}
```

La sortie est :

```
3
3
3
```

Le test facultatif suivant compile :

```
const double& r4{4.0};
```

Une référence constante peut aussi désigner une valeur temporaire. Ce cas sera approfondi ultérieurement.

À retenir. Une référence `const` permet de lire l'objet, mais pas de le modifier via cette référence.

Exercice 6 — 0 ou nullptr ?

```
#include <iostream>

void inspect(int value) {
    std::cout << "inspect(int): " << value << '\n';
}

void inspect(double* values) {
    std::cout << "inspect(double*): " << values << '\n';
}

int main() {
    inspect(0);
    inspect(nullptr);
}
```

Sortie typique :

```
inspect(int): 0
inspect(double*): 0
```

La représentation affichée pour le pointeur nul peut aussi être `0x0`.

1. `inspect(0)` appelle `inspect(int)` : 0 est un littéral entier.
2. `inspect(nullptr)` appelle `inspect(double*)`.
3. `nullptr` exprime clairement l'intention « aucun objet pointé ».

Dans du vieux code C ou C++, on rencontre également `NULL`. En C++ moderne, on préfère `nullptr`.

À retenir. `nullptr` représente explicitement un pointeur nul.

Synthèse

Écriture	Signification
<code>double x{};</code>	<code>x</code> est initialisé à 0.0
<code>int x{2.9};</code>	conversion réductrice refusée
<code>double& r{x};</code>	<code>r</code> est un autre nom pour <code>x</code>
<code>const double& r{x};</code>	lecture possible, modification interdite via <code>r</code>
<code>Sample& s</code>	la fonction peut modifier l'objet de l'appelant
<code>const Sample& s</code>	la fonction lit l'objet sans le modifier via <code>s</code>
<code>double x</code>	le paramètre reçoit sa propre valeur
<code>nullptr</code>	pointeur nul explicite
<code>std::vector<int></code>	conteneur qui gère lui-même son stockage

Pour lire une signature, il faut pouvoir répondre à une question simple : la fonction reçoit-elle une valeur indépendante, peut-elle modifier l'objet fourni ou peut-elle seulement le lire ?